



南開大學
Nankai University

计算机学院
数据安全实验报告

秘密共享实践

姓名：王一诺
学号：2312331
专业：计算机科学与技术

2026 年 5 月 24 日

目录

1	实验要求	2
2	实验原理	2
2.1	秘密共享	2
2.2	Shamir 门限秘密共享	2
2.3	基于秘密共享的平均值计算	3
3	实验过程	4
3.1	实验代码说明	4
3.2	生成三个同学的秘密份额	4
3.3	各同学计算份额和	6
3.4	重构总和并计算平均值	7
4	结论与实践心得	8

1 实验要求

假设有三个同学需要对班里的优秀干部 Alice、Bob、Charles、Douglas 进行投票，最后统计各个班干部获得的票数。这个时候就可以利用 Shamir 秘密共享将各个投票方的投票分享出去并进行隐私求和计算。

借鉴实验 3.2，实现三个人对于他们拥有的数据的平均值的计算。

2 实验原理

2.1 秘密共享

秘密共享 (Secret Sharing) 是一类将秘密拆分为多个份额并分发给不同参与方的密码技术。单个份额不应泄露原始秘密，只有满足重构条件的参与方集合才能恢复秘密。其核心目标是避免秘密集中存放带来的单点泄露风险，同时支持后续的安全多方计算。

在本实验中，每个同学持有一个私有数据。为了在不直接公开各自数据的情况下计算平均值，每个同学先把自己的数据拆成多个秘密份额，再把份额分发给其他同学。随后各同学只在份额层面做加法，最终由计数方根据足够数量的份额重构总和，再除以人数得到平均值。

2.2 Shamir 门限秘密共享

Shamir 秘密共享是一种典型的 (t, n) 门限秘密共享方案。其思想是利用多项式插值：次数为 $t-1$ 的多项式由 t 个不同点唯一确定，而少于 t 个点无法确定该多项式的常数项。

设秘密为 s ，在有限域 $\mathbb{Z}_p$ 上随机选择一个次数为 $t-1$ 的多项式：

$$f(x) = s + a_1x + a_2x^2 + \cdots + a_{t-1}x^{t-1} \pmod{p}$$

其中 $f(0) = s$ ，随机系数 $a_1, \dots, a_{t-1}$ 用于隐藏秘密。对不同的非零横坐标 x_i 计算 $f(x_i)$ ，得到秘密份额 $(x_i, f(x_i))$ 。重构时使用拉格朗日插值在 $x = 0$ 处求值：

$$f(0) = \sum_{i=1}^t y_i \prod_{j \neq i} \frac{0 - x_j}{x_i - x_j} \pmod{p}$$

在模素数 p 下，除法通过乘以模逆元实现。参考代码中的 `quickpower` 使用快速幂计算 $b^{p-2} \bmod p$ ，从而依据费马小定理得到 b 的模逆。

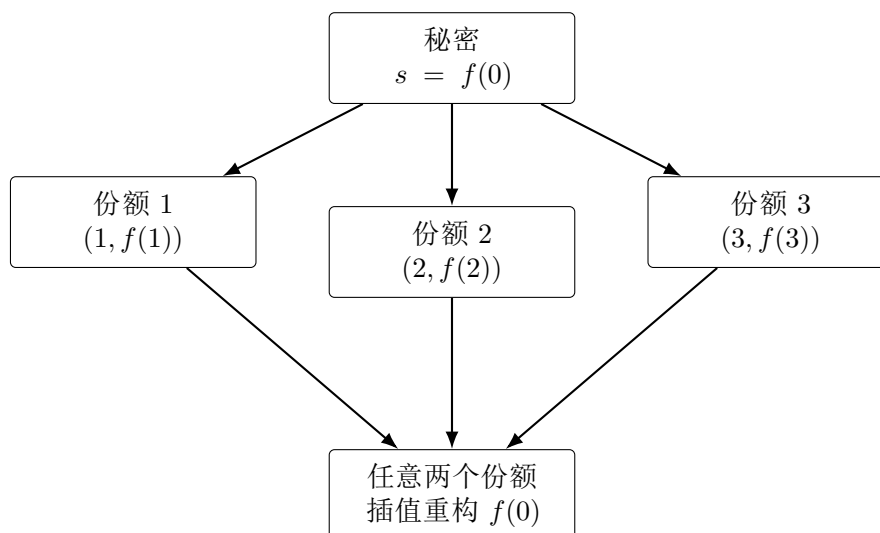


图 2.1: Shamir 门限秘密共享基本原理

2.3 基于秘密共享的平均值计算

本实验采用 (2,3) 门限方案，即共有 3 个参与方，任意 2 个求和份额即可重构结果。因此每个同学构造一次一阶多项式：

$$f_i(x) = s_i + r_i x \pmod{p}, \quad i = 1, 2, 3$$

其中 s_i 是第 i 个同学的私有数据， r_i 是随机数。第 i 个同学分别计算 $f_i(1), f_i(2), f_i(3)$ ，并将第 j 个份额发送给 Student j 。

Student j 收到三个份额后，只计算份额和：

$$d_j = f_1(j) + f_2(j) + f_3(j) \pmod{p}$$

由于多项式加法满足线性性质， d_j 实际上是总和多项式 $F(x) = f_1(x) + f_2(x) + f_3(x)$ 在 $x = j$ 处的取值，而

$$F(0) = f_1(0) + f_2(0) + f_3(0) = s_1 + s_2 + s_3$$

因此计数方只需任选两个 d_j 进行插值重构，就能得到三个数据的总和，再除以 3 得到平均值：

$$\bar{s} = \frac{F(0)}{3}$$

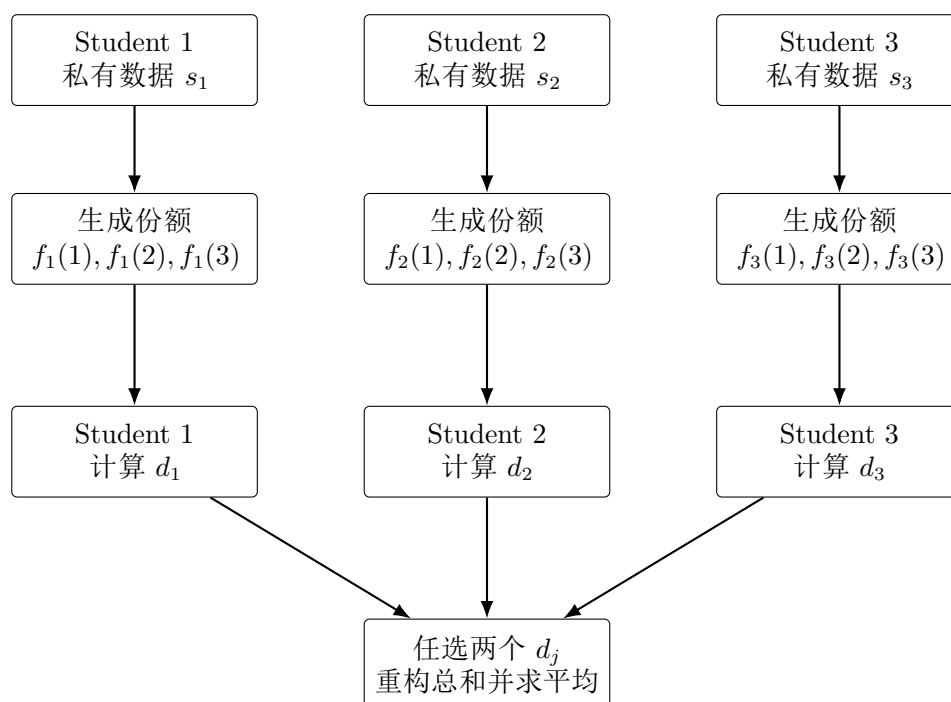


图 2.2: 基于 Shamir 秘密共享的隐私平均值计算流程

3 实验过程

3.1 实验代码说明

本次实验保持参考代码内容完全一致,不对 `ss_function.py`、`ss_student.py`、`count_student.py` 和 `vote_counter.py` 进行修改。四个文件的作用如下:

- `ss_function.py`: 实现快速幂、随机多项式生成、多项式取值和拉格朗日插值重构;
- `ss_student.py`: 每个同学输入自己的数据, 构造一阶多项式并生成三个秘密份额;
- `count_student.py`: 每个同学读取属于自己的三个份额并求和, 生成 `d_i.txt`;
- `vote_counter.py`: 计数方读取两个求和份额, 重构总和并除以 3 得到平均值。

其中 `vote_counter.py` 中的关键平均值计算语句为:

```

1 d=ss_f.restructure_polynomial([2,3],d_23,2,p)
2 # 这一步是为了计算平均值
3 d = d / 3
4 print(f'得票结果为: {d}')
```

虽然输出提示文字仍保留为“得票结果”,但此时变量 `d` 已经由总和除以 3, 因此输出值即为三个输入数据的平均值。

3.2 生成三个同学的秘密份额

本实验沿用投票场景中的二值数据, 令三个同学对某一候选人的输入分别为:

$$s_1 = 0, \quad s_2 = 1, \quad s_3 = 0$$

因此明文平均值应为 $(0 + 1 + 0)/3 = 0.333333\dots$ 。在不公开各自输入的情况下，三个同学分别执行 `ss_student.py` 生成秘密份额。

```
D:\数据安全\labcode\lab3\参考\codes>python ss_student.py
模数 p: 1000000007
请输入参与方 id:1
请输入 student_1的投票值 s:0
Student_1的投票值的多项式及秘密份额:
f1=0+481449096x^1
(1,481449096)
(2,962898192)
(3,444347281)

D:\数据安全\labcode\lab3\参考\codes>python ss_student.py
模数 p: 1000000007
请输入参与方 id:2
请输入 student_2的投票值 s:1
Student_2的投票值的多项式及秘密份额:
f2=1+538118585x^1
(1,538118586)
(2,76237164)
(3,614355749)

D:\数据安全\labcode\lab3\参考\codes>python ss_student.py
模数 p: 1000000007
请输入参与方 id:3
请输入 student_3的投票值 s:0
Student_3的投票值的多项式及秘密份额:
f3=0+716816366x^1
(1,716816366)
(2,433632725)
(3,150449084)
```

图 3.3: 三个同学生成秘密份额

本机一次运行得到的随机多项式和份额如下。由于多项式斜率由 `random.randrange(0,p)` 随机生成，每次运行的中间份额会不同，但最终平均值应保持一致。

表 1: 三个同学的秘密份额生成结果

参与方	输入数据	随机多项式	$x = 1$ 份额	$x = 2$ 份额	$x = 3$ 份额
Student 1	0	$f_1(x) = 0 + 481449096x$	481449096	962898192	444347281
Student 2	1	$f_2(x) = 1 + 538118585x$	538118585	76237164	614355749
Student 3	0	$f_3(x) = 0 + 716816366x$	716816366	433632725	150449084

运行完成后,程序生成 `student_1_1.txt` 到 `student_3_3.txt` 共 9 个份额文件。文件名 `student_i_j.txt` 表示 Student i 为 Student j 生成的秘密份额。

student_1_1	2026/5/24 22:01	文本文档
student_1_2	2026/5/24 22:01	文本文档
student_1_3	2026/5/24 22:01	文本文档
student_2_1	2026/5/24 22:03	文本文档
student_2_2	2026/5/24 22:03	文本文档
student_2_3	2026/5/24 22:03	文本文档
student_3_1	2026/5/24 22:03	文本文档
student_3_2	2026/5/24 22:03	文本文档
student_3_3	2026/5/24 22:03	文本文档

图 3.4: 生成的 9 个秘密份额文件

3.3 各同学计算份额和

接下来三个同学分别执行 `count_student.py`。Student j 读取 `student_1_j.txt`、`student_2_j.txt`、`student_3_j.txt`, 并计算:

$$d_j = f_1(j) + f_2(j) + f_3(j) \pmod{p}$$

该过程只在秘密份额上做加法, 不需要知道其他同学的原始输入。

```
D:\数据安全\labcode\lab3\参考\codes>python count_student.py
请输入参与方 id:1

D:\数据安全\labcode\lab3\参考\codes>python count_student.py
请输入参与方 id:2

D:\数据安全\labcode\lab3\参考\codes>python count_student.py
请输入参与方 id:3
```

图 3.5: 三个同学计算秘密份额和

本机一次运行生成的三个求和份额如下：

$$d_1 = 736384041, \quad d_2 = 472768074, \quad d_3 = 209152107$$

三个求和结果分别写入 `d_1.txt`、`d_2.txt`、`d_3.txt`。

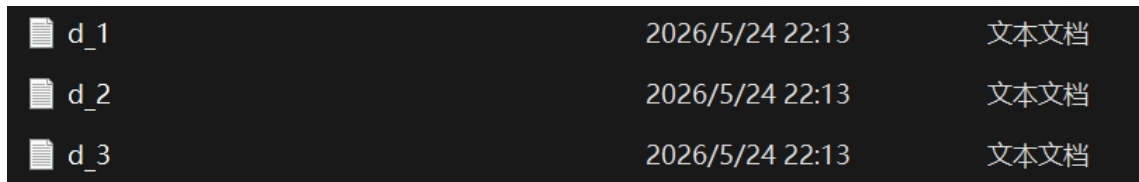


图 3.6: 生成的求和份额文件

3.4 重构总和并计算平均值

最后计数方执行 `vote_counter.py`。该程序读取 `d_2.txt` 和 `d_3.txt`，使用两个点 $(2, d_2)$ 、 $(3, d_3)$ 对总和多项式 $F(x)$ 做拉格朗日插值，恢复 $F(0)$ 。

在本次运行中，总和多项式的一次项系数为：

$$481449096 + 538118585 + 716816366 \equiv 736384041 \pmod{1000000007}$$

所以总和多项式可表示为：

$$F(x) = 1 + 736384041x \pmod{1000000007}$$

对应的 $F(2) = 472768074$ 、 $F(3) = 209152107$ 与文件中的 `d_2.txt`、`d_3.txt` 一致。通过插值恢复得到：

$$F(0) = 1$$

因此平均值为：

$$\bar{s} = \frac{1}{3} = 0.3333333333333333$$

```
D:\数据安全\labcode\lab3\参考\codes>python count_student.py
请输入参与方 id:1

D:\数据安全\labcode\lab3\参考\codes>python count_student.py
请输入参与方 id:2

D:\数据安全\labcode\lab3\参考\codes>python count_student.py
请输入参与方 id:3

D:\数据安全\labcode\lab3\参考\codes>python vote_counter.py
得票结果为: 0.3333333333333333
```

图 3.7: 重构并输出平均值

程序最终输出：

```
1 得票结果为：0.3333333333333333
```

```
2
```

结果与明文直接计算的 $(0 + 1 + 0)/3$ 一致，说明利用 Shamir 秘密共享可以在不公开各同学原始数据的前提下完成平均值计算。

4 结论与实践心得

通过实验可以看到，Shamir 秘密共享不仅可以恢复单个秘密，也可以利用其线性性质完成隐私求和。每个参与方只暴露多项式取值形式的份额，计数方只获取两个求和后的份额，因此不会直接得到某个同学的原始输入。在本实验采用的 $(2, 3)$ 门限设置下，任意两个求和份额即可重构总和，少于两个份额则无法确定总和多项式。

本实验也体现了秘密共享方案的局限：参与方数量和门限值需要提前确定；协议要求各参与方正确保存和传递份额；如果扩展到更复杂的数据类型或更大规模参与方，还需要考虑通信开销、恶意参与方校验和可验证秘密共享等问题。总体来看，Shamir 秘密共享结构清晰、实现成本较低，是理解安全多方计算中隐私求和与平均值计算的重要基础。